RAGForge: Project Report

Nov 10, 2026 · @Gunii

1. Executive summary

RAGForge is a retrieval-augmented generation engine whose core parts (chunker, BM25 index, vector index, rank fusion, evaluator) are written by hand, then benchmarked against FAISS and managed services to show measured trade-offs.

Why from scratch. Wiring LangChain to a vector database proves you can follow a tutorial. Building the retrieval internals and defending every design choice with numbers proves you can reason about systems, which is what Amazon-style interviews test.

Goals

- Build a hybrid (dense + keyword) retrieval engine with reranking, citations and a groundedness check.
- Ship it as a deployed, observable API on AWS with CI and an evaluation gate.
- Publish an ablation table showing what each component adds in quality, latency and cost.

Success metrics

Metric	Target
Recall@5 on own eval set	0.85 or higher (dense-only baseline recorded first)
Faithfulness (LLM-judge plus spot checks)	0.90 or higher
p95 end-to-end latency (retrieval + rerank, excluding LLM)	under 400 ms on 100k chunks
Own HNSW index vs FAISS HNSW	within 5 points of recall at the same ef
Cost per 1,000 queries	measured and reported

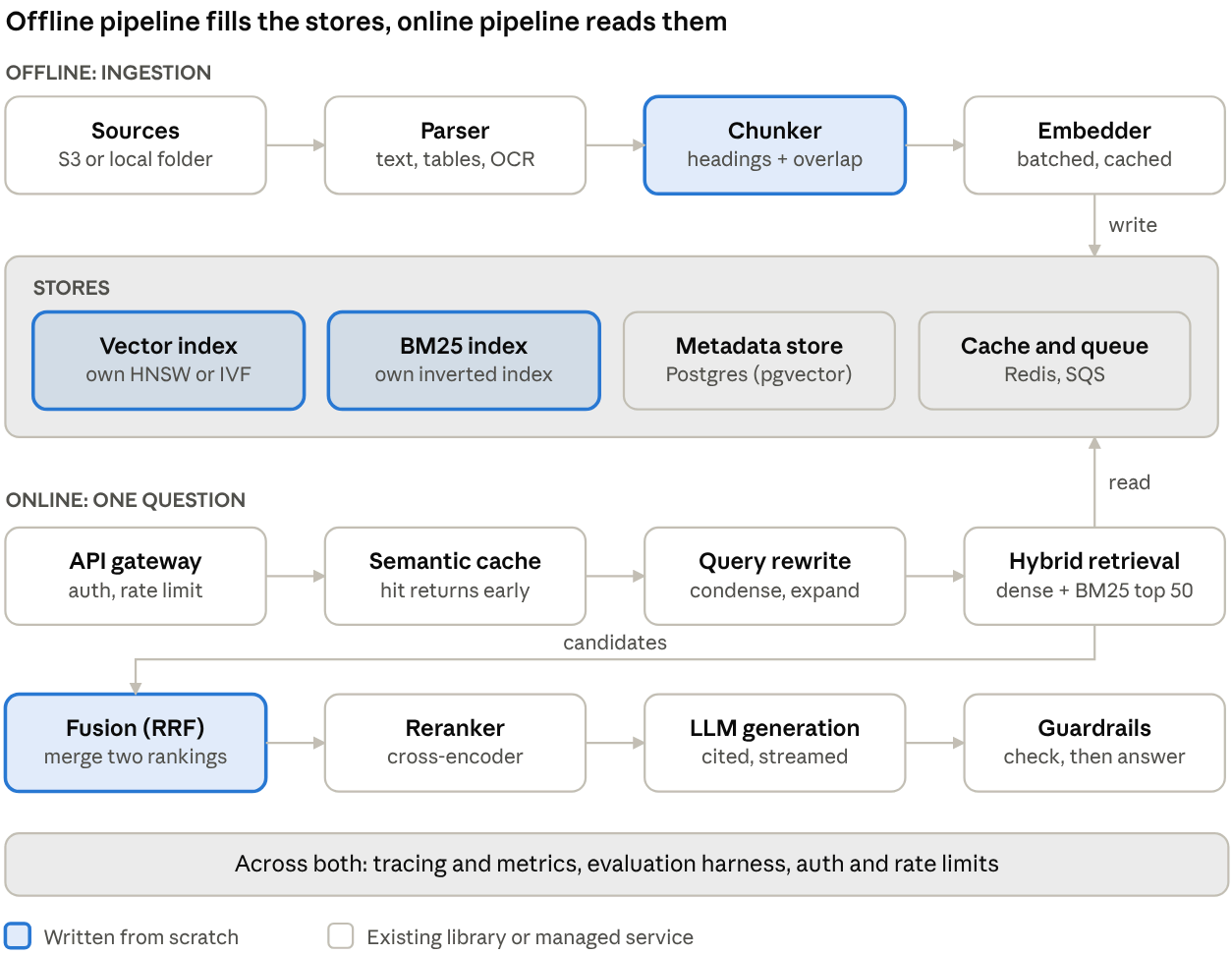
2. Problem statement and scope

Large language models do not know private or recent documents and will invent answers when asked about them; RAG fixes this by retrieving relevant passages first and making the model answer only from them. Most student RAG projects stop at "load PDF, embed, ask question" with no measurement of whether retrieval or answers are any good. RAGForge closes that gap.

Suggested corpus. Use a public technical corpus of roughly 100k chunks, for example openly published cloud documentation plus a set of research papers, so the demo speaks directly to the target employer and the answers can be checked by anyone.

In scope	Out of scope
Ingestion of PDF, Markdown, HTML and plain text	Training your own LLM or embedding model
Hybrid retrieval, reranking, cited answers	A polished front end (a minimal chat page is enough)
Custom BM25 and HNSW or IVF index	Multi-tenant billing and user management
Evaluation harness, tracing, CI gate	Multi-modal retrieval (images, audio); listed as a stretch goal
Deployment on AWS with cost report	Fine-tuning the reranker (optional stretch)

3. System architecture



architecture · 2 pipelines, 4 stores

Documents flow through four steps into the stores; each question then flows through retrieval, fusion, reranking, generation and a guardrail check. Highlighted boxes are the parts you write yourself.

Layer	Responsibility	Key design decision
Ingestion	Turn raw files into clean, enriched chunks	Heading-aware chunking with parent ids
Indexing and storage	Hold vectors, keywords, text, metadata, caches	Own indexes behind an interface so FAISS or OpenSearch can be swapped in for benchmarks

Layer	Responsibility	Key design decision
Retrieval	Find candidate chunks for a query	Hybrid dense and BM25, fused by rank
Ranking	Improve precision of the top results	Cross-encoder on the top 20 only
Generation	Produce a cited answer from context	Token budget, citation ids, abstain when evidence is thin
Safety	Stop injection, leakage and unsupported claims	Groundedness check and access filter applied before ranking
Platform	Serve, observe, evaluate, deploy	Streaming API, tracing per stage, CI evaluation gate

4. Workflows

There are two pipelines: an offline one that turns documents into searchable indexes, and an online one that answers each question. A third loop uses logged traces to improve both.

4.1 Offline ingestion workflow

1. **Collect.** Read files from S3 or a local folder; record source, version and a content hash for each.
2. **Parse.** Extract text, headings, tables and page numbers; fall back to OCR for scanned pages.
3. **Clean and deduplicate.** Strip boilerplate, normalise whitespace, drop exact duplicates by hash and near-duplicates by MinHash.
4. **Chunk.** Split by headings first, then recursively by size with overlap; keep a parent chunk id so a small matched chunk can return its larger parent.
5. **Enrich.** Attach metadata (source, section path, page, date, document type) used later for filtering and citations.
6. **Embed.** Batch chunks through the embedding model asynchronously; cache vectors by content hash so unchanged text is never re-embedded.
7. **Index.** Write vectors to the vector index, tokens to the BM25 inverted index, and chunk text plus metadata to the document store.

8. **Verify.** Run a sample-query smoke test and log counts (documents, chunks, tokens, failures) before the new index version goes live.

Incremental updates: a changed hash re-runs steps 2 to 7 for that document only and deletes its old chunks.

4.2 Online query workflow

1. **Receive** the request (query, optional filters, optional conversation id) and authenticate it.
2. **Check the semantic cache.** A near-identical earlier query returns its stored answer immediately.
3. **Understand the query.** Condense chat history into a standalone question, then optionally expand it (multi-query or HyDE).
4. **Retrieve in parallel.** Dense top 50 from the vector index and BM25 top 50 from the keyword index, with metadata filters applied.
5. **Fuse.** Combine both lists with Reciprocal Rank Fusion into one ranked list.
6. **Rerank.** A cross-encoder scores the top 20 fused candidates against the query; keep the best 5.
7. **Build context.** Swap child chunks for parent chunks where useful, order by relevance, and trim to the token budget.
8. **Generate.** Prompt the LLM to answer only from the numbered context and to cite chunk ids; stream tokens to the client.
9. **Check.** A groundedness check tests that each claim is supported; on failure retry once with more context, otherwise answer that the documents do not contain enough information.
10. **Respond and log.** Return the answer with citations and write a trace (stage latencies, retrieved ids, scores, tokens, cost).

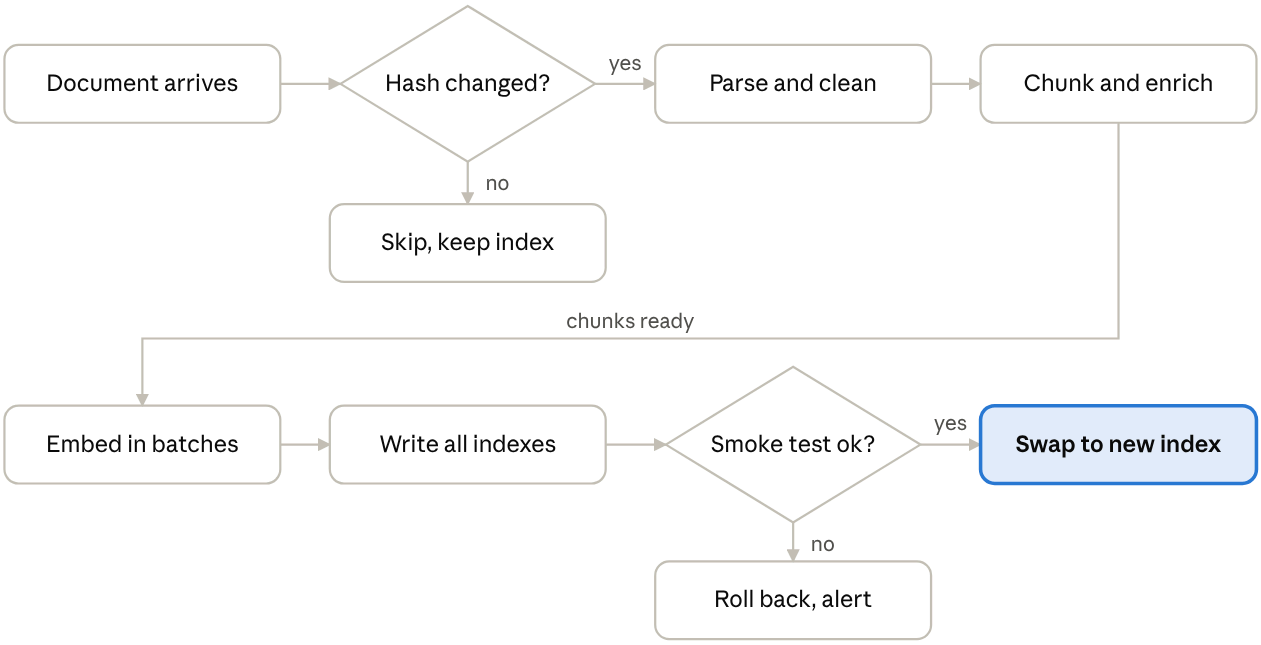
4.3 Feedback and improvement loop

1. Collect thumbs up or down and flagged answers from the API.
2. Add failed queries, with their correct source chunks, to the evaluation set.
3. Re-run the evaluation suite on every change; a drop in Recall@5 or faithfulness blocks the merge.

5. Flowcharts

5.1 Ingestion flow

Unchanged documents are skipped; a failed smoke test never goes live

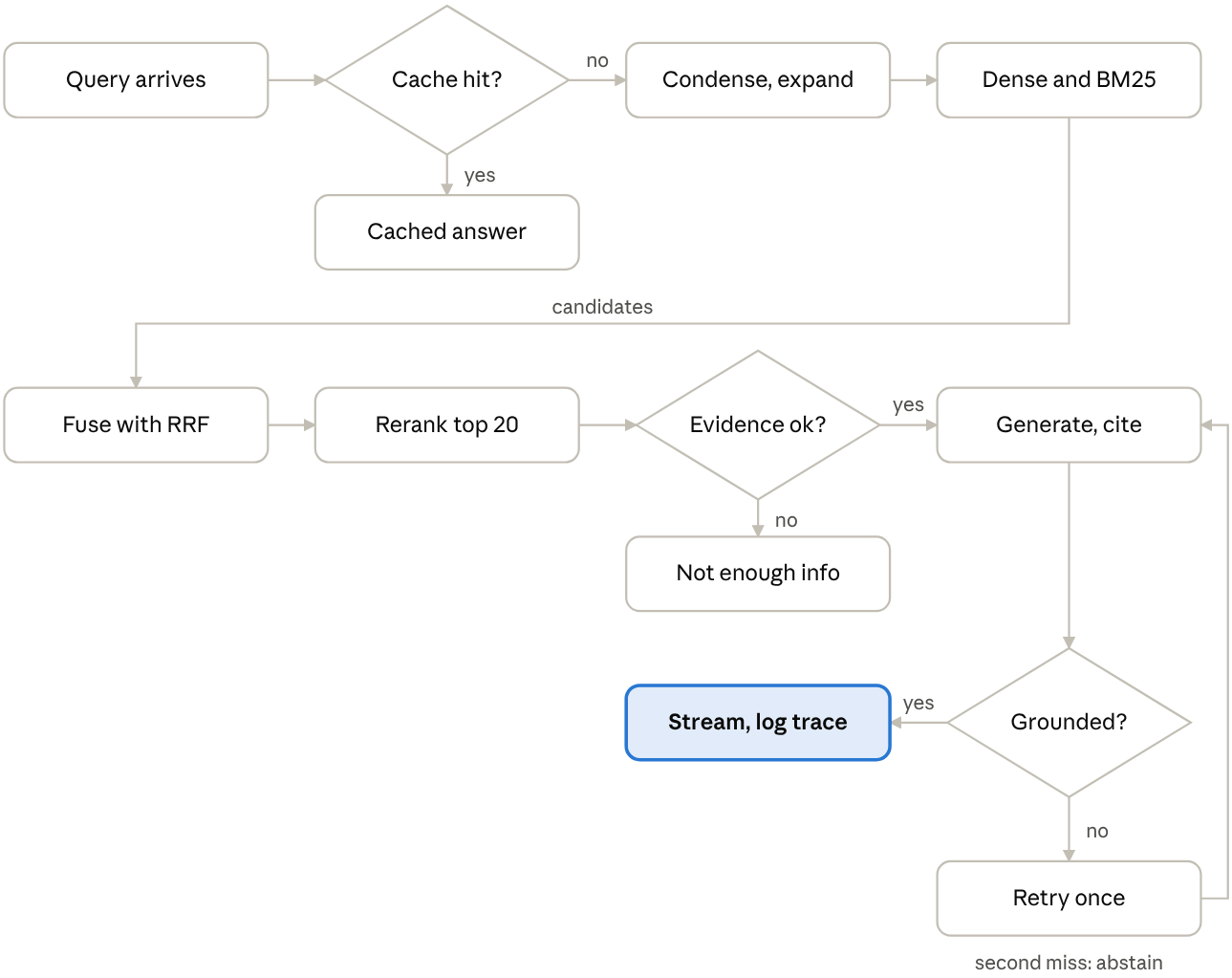


ingestion flow · 2 decisions

A content hash decides whether a document is touched at all, and a new index version goes live only after its smoke test passes.

5.2 Query flow

A cited answer streams only after evidence and grounding checks pass



query flow · 3 decisions, 1 retry loop

Three gates shape every query: a cache hit returns early, weak evidence ends in an honest refusal, and an ungrounded draft gets one retry before the system abstains.

6. Features

Must-have features make a working, measurable engine; advanced features are what separate this from a tutorial project. Build in priority order.

Feature	What it does	Level
Multi-format ingestion	PDF, Markdown, HTML, text with page and section metadata	Must
Recursive and heading-aware chunking	Chunks that respect document structure, with overlap	Must
Embedding cache	Skips re-embedding unchanged text via content hash	Must
BM25 keyword search (own code)	Inverted index with tuned k1 and b	Must
Vector search (own code)	Brute-force baseline, then HNSW or IVF	Must
Hybrid retrieval with RRF	Merges dense and keyword rankings	Must
Cited answers	Every claim links to a chunk id, page and source	Must
Evaluation harness	Recall@k, MRR, nDCG, faithfulness, ablation table	Must
Cross-encoder reranking	Re-scores top candidates for precision	Advanced
Parent-child retrieval	Match on small chunks, answer from larger context	Advanced
Query rewriting, multi-query, HyDE	Handles vague, multi-part and follow-up questions	Advanced
Metadata filtering	Restrict by source, date, document type	Advanced
Groundedness check and abstention	Refuses to answer when evidence is missing	Advanced
Semantic cache	Reuses answers for near-duplicate queries	Advanced
Incremental indexing	Re-processes only changed documents	Advanced
Streaming API with tracing	Server-sent events, per-stage latency and cost	Advanced
Prompt-injection and PII defences	Sanitises retrieved text and redacts personal data	Advanced

Feature	What it does	Level
Index versioning and rollback	Blue-green index swap with a smoke test	Stretch
Context compression	Drops low-value sentences to save tokens	Stretch
Multi-modal retrieval	Images and tables via captioning or CLIP	Stretch
Agentic retrieval	Model decides when to search again or call tools	Stretch

7. Databases and storage options

Use your own index as the primary engine and one managed or open-source store as the comparison backend; a pgvector setup is the simplest way to cover metadata and vectors together in production.

7.1 Vector stores

Option	Type	Strengths	Trade-offs	Role in RAGForge
Own HNSW / IVF (NumPy or C++)	Library you write	Full control, best interview story	You own correctness and speed	Primary engine
FAISS	Library	Fast, many index types, easy benchmark	No built-in persistence, filtering or server	Benchmark baseline
pgvector (Postgres)	Extension	Vectors, metadata and SQL filters in one place	Slower than specialised engines at very large scale	Recommended production store
Amazon OpenSearch Service	Managed search	Hybrid (BM25 plus k-NN) built in, AWS-native	Cost; more setup	AWS comparison backend
Qdrant	Vector database	Strong filtering, simple ops, open source	Another service to run	Optional comparison

Option	Type	Strengths	Trade-offs	Role in RAGForge
Chroma	Embedded database	Very quick to start	Less suited to production scale	Prototyping only
Pinecone	Managed service	No operations work	Vendor lock-in, cost	Mention in trade-off discussion

7.2 Other stores

Data	Store	Why
Chunk text and metadata	PostgreSQL (or SQLite for local dev)	Relational filters, transactions, easy joins to document records
Keyword index	Own inverted index on disk, or OpenSearch for comparison	Needed for BM25
Embedding and answer cache	Redis (Amazon ElastiCache)	Low-latency lookups and expiry
Raw documents and index snapshots	Amazon S3	Cheap, durable, versioned
Traces, metrics, eval runs	PostgreSQL tables plus CloudWatch or Prometheus	Query history and dashboards
Ingestion job queue	Amazon SQS (or Redis queue locally)	Decouples ingestion from the API

7.3 Core tables

Table	Key columns
documents	doc_id, source_uri, content_hash, version, ingested_at
chunks	chunk_id, doc_id, parent_chunk_id, text, section_path, page, token_count
embeddings	chunk_id, model_name, vector (or file offset), created_at

Table	Key columns
queries	query_id, text, filters, created_at, user_feedback
traces	query_id, stage, latency_ms, tokens_in, tokens_out, cost_usd, retrieved_chunk_ids
eval_cases	case_id, question, gold_answer, gold_chunk_ids
eval_runs	run_id, git_sha, config_json, recall_at_5, mrr, faithfulness, p95_latency_ms

8. Technology stack

Layer	Choice	Alternatives
Language	Python 3.11 (C++ via pybind11 for the index hot loop, optional)	Go or Rust for the serving layer
API	FastAPI, Pydantic, Uvicorn, server-sent events	Flask, gRPC
Parsing	PyMuPDF, unstructured, BeautifulSoup, Tesseract OCR	Amazon Textract
Embeddings	BGE or E5 via sentence-transformers (open, runs locally)	Amazon Titan Embeddings, OpenAI embeddings
Reranker	bge-reranker cross-encoder	Cohere Rerank, a small ColBERT-style model
LLM	Any hosted model behind one interface (Amazon Bedrock, Claude, OpenAI), plus a small local model for offline tests	Llama or Mistral via vLLM
Index	Own BM25 and HNSW or IVF; FAISS and pgvector as baselines	OpenSearch, Qdrant
Storage	PostgreSQL, Redis, S3	DynamoDB for traces
Evaluation	Own harness plus RAGAS or DeepEval for cross-checking	TruLens

Layer	Choice	Alternatives
Observability	OpenTelemetry, Prometheus, Grafana or CloudWatch	Langfuse for LLM traces
Packaging and CI	Docker, GitHub Actions, pytest	Make, pre-commit
Cloud	AWS: S3, ECS Fargate or Lambda, ElastiCache, RDS, CloudWatch, Bedrock	Any other cloud; the design is portable

9. Component deep dives

9.1 Chunking

Chunk size trades precision against context: small chunks match queries sharply but lose surrounding meaning, large chunks do the opposite. Start with heading-aware splitting, then recursive splitting at 300 to 500 tokens with 10 to 15 percent overlap, and tune on your evaluation set instead of guessing. Parent-child retrieval removes most of the dilemma: index small child chunks, return the larger parent for the prompt.

9.2 Embeddings

Wrap the embedding model behind one interface so models can be swapped and compared. Normalise vectors so cosine similarity becomes a dot product, batch requests, and key the cache on a hash of the text plus the model name. Record the model name with every vector; mixing models in one index silently breaks retrieval.

9.3 BM25 (write it yourself)

BM25 scores a document by how often query terms occur in it, discounted by how common each term is across the corpus and by document length:

```
\text{score}(D,Q)=\sum_{t\in Q}\text{IDF}(t)\cdot\frac{f(t,D)\,(k_1+1)}
{f(t,D)+k_1\left(1-b+b\,\frac{|D|}{\text{avgdl}}\right)}
```

Build an inverted index (term to list of chunk ids and term frequencies), store document lengths, and tune k1 (typically 1.2 to 2.0) and b (about 0.75) on your evaluation set. BM25 wins on exact identifiers, error codes and rare names, where embeddings are weakest.

9.4 Vector index (write it yourself)

Index	How it works	Strength	Cost
Brute force	Compare the query with every vector	Exact; the ground truth for recall	Linear in corpus size
IVF	Cluster vectors with k-means, search only the nearest clusters (nprobe)	Simple, memory-friendly	Recall drops if nprobe is low
HNSW	Layered proximity graph; greedy descent from sparse top layers to dense bottom layer	Best recall-versus-speed trade-off	Higher memory; slower inserts
Product quantisation	Compress vectors into short codes	Large memory savings	Some recall loss

Implement brute force first and use it to measure the recall of every approximate index. Report recall versus latency while sweeping ef (HNSW) or nprobe (IVF), and compare your curve with FAISS.

9.5 Hybrid fusion

Dense and keyword scores are on different scales, so fuse ranks, not scores. Reciprocal Rank Fusion adds 1 / (k + rank) across lists, with k around 60:

```
\text{RRF}(d)=\sum_{r\in R}\frac{1}{k+\text{rank}_r(d)}
```

9.6 Reranking

A bi-encoder embeds query and passage separately, which is fast but coarse. A cross-encoder reads them together and scores relevance more accurately but costs one model pass per candidate, so apply it only to the top 20 to 50 fused results. Measure the quality gain against the added milliseconds and report both.

9.7 Query understanding

- **Condense:** rewrite a follow-up such as "what about pricing?" into a standalone question using chat history.

- **Multi-query:** generate several paraphrases, retrieve for each, and fuse the lists.
- **HyDE:** have the model write a hypothetical answer and embed that, which often matches documents better than the short question does.
- **Routing:** send simple lookups to keyword search and conceptual questions to dense plus rerank.

9.8 Generation

Number each context chunk and instruct the model to answer only from them, cite by number, and say so when the context lacks the answer. Budget tokens explicitly (system prompt, context, question, answer). Put the strongest chunks at the start and end of the context, because models attend less to the middle.

10. Evaluation framework

The evaluation harness is the most important deliverable: it turns opinions about design into numbers and blocks regressions.

Dataset. Build 150 to 200 cases of question, reference answer and gold source chunks. Generate candidates with an LLM from sampled chunks, then verify each by hand; include easy lookups, multi-hop questions, ambiguous questions, and 20 or more questions the corpus cannot answer (to test abstention).

Layer	Metric	What it tells you
Retrieval	Recall@k (k = 1, 5, 10)	Was the gold chunk retrieved at all?
Retrieval	MRR and nDCG@10	How high was it ranked?
Generation	Faithfulness	Are all claims supported by the retrieved context?
Generation	Answer relevance and correctness	Does the answer address the question, and match the reference?
Generation	Citation accuracy	Do cited chunks actually support the cited claims?
Safety	Abstention rate on unanswerable questions	Does it refuse instead of inventing?

Layer	Metric	What it tells you
System	p50 and p95 latency per stage, cost per query, throughput	Can it run in production?

Ablation plan. Run the same cases after adding one component at a time and record every metric:

1. Dense only, brute force.
2. Dense with own HNSW (measures index recall loss).
3. BM25 only.
4. Hybrid with RRF.
5. Hybrid plus reranker.
6. Hybrid plus reranker plus query rewriting.
7. Chunk size and overlap sweep on the best configuration.

Judging. Use an LLM judge for faithfulness and relevance, but check its agreement against 30 to 50 hand-labelled answers and report that agreement; an unvalidated judge is a weak result.

CI gate. A GitHub Actions job runs a small fixed subset of cases on every pull request and fails the build if Recall@5 or faithfulness drops by more than a set margin versus main. Store each run in the eval_runs table with the git commit.

11. Security, guardrails and responsible AI

Risk	Mitigation
Indirect prompt injection (instructions hidden inside retrieved documents)	Wrap context in delimiters, tell the model to treat it as data, strip instruction-like patterns, never give the model tools that act on retrieved text without confirmation
Hallucination	Groundedness check, mandatory citations, abstain when retrieval scores fall below a threshold
PII leakage	Detect and redact emails, phone numbers and IDs at ingestion and in responses (Presidio or regex plus allow-lists)

Risk	Mitigation
Unauthorised document access	Store an access-control list per document, filter retrieval by the caller's groups before ranking, never after
Abuse and cost blow-ups	API keys, per-key rate limits, request size caps, token budgets, timeouts
Secrets and data in transit	AWS Secrets Manager, TLS everywhere, S3 encryption at rest, least-privilege IAM roles
Stale or wrong sources	Source and date metadata in every citation, index versioning, scheduled re-ingestion
Unsafe output	Output moderation pass, refusal behaviour tested in the evaluation set

Include a short "threat model" page in the repository naming these risks; it shows engineering maturity beyond accuracy numbers.

12. Scalability and performance

Build and benchmark at 100k chunks, then write a design note on how each part would change at 100 million. Interviewers ask this question; having reasoned answers is the goal, not running the full scale.

Concern	At 100k chunks (built)	At 100M chunks (design note)
Vector index size	Single in-memory HNSW	Shard by document hash or tenant; product quantisation; disk-based indexes such as DiskANN-style graphs; tiered hot and cold storage
Embedding throughput	Batched async calls with cache	Distributed workers reading from SQS, GPU batch inference, backpressure and retries with dead-letter queue
Query fan-out	One index	Scatter-gather across shards, merge top-k, tail-latency control with hedged requests and timeouts

Concern	At 100k chunks (built)	At 100M chunks (design note)
Index updates	Rebuild or incremental insert	Write-ahead log, segment merges, blue-green index swap
Reranking cost	Rerank top 20	Cascade: cheap filter first, rerank fewer candidates, cache hot queries
LLM cost and latency	Streaming, token budget	Prompt caching, smaller model for easy queries, answer cache
Availability	Single service	Multi-AZ, health checks, autoscaling on queue depth and p95 latency, graceful degradation to keyword-only search if the vector index is down

Performance habits to show. Profile before optimising; run retrieval branches concurrently with asyncio; batch and vectorise NumPy operations; keep the model loaded in memory; set per-stage timeouts; report p50 and p95, not averages.

13. Deployment on AWS, CI/CD and cost

Component	AWS service	Notes
Raw documents and index snapshots	S3 (versioned, encrypted)	Source of truth; an S3 event triggers ingestion
Ingestion jobs	SQS plus ECS Fargate workers	Autoscale on queue depth
Query API	ECS Fargate behind an Application Load Balancer (or Lambda for a cheaper demo)	Container built from the repository Dockerfile
Metadata and vectors	RDS PostgreSQL with pgvector	Optional OpenSearch domain for the comparison run
Cache	ElastiCache for Redis	Embedding and semantic cache
LLM and embeddings	Amazon Bedrock, or an external API	Behind a provider interface
Secrets	Secrets Manager	No keys in code or images

Component	AWS service	Notes
Logs, metrics, alarms	CloudWatch plus OpenTelemetry	Latency and error-rate alarms
Infrastructure as code	Terraform or AWS CDK	Whole stack reproducible with one command

CI/CD pipeline (GitHub Actions).

1. Lint, type-check and unit tests on every push.
2. Evaluation subset on every pull request; fail on quality regression.
3. Build and scan the Docker image.
4. Deploy to a staging service, run a smoke test, then promote to production.
5. Index builds run as a separate versioned job; the API switches to the new index only after its smoke test passes, and can roll back to the previous snapshot.

Cost. Track cost per 1,000 queries as a line in the evaluation report, broken down into embedding, reranking, LLM tokens and infrastructure. Keep the demo within a small monthly budget by using Fargate Spot or Lambda, scaling to zero when idle, and tearing down OpenSearch after the benchmark run.

14. Observability and monitoring

Every request gets a trace id, and each pipeline stage is a span, so any slow or wrong answer can be replayed from its logged evidence.

Signal	What to record	Use
Latency	Per-stage p50 and p95 (cache, rewrite, retrieve, fuse, rerank, generate)	Find the slow stage
Retrieval quality	Retrieved chunk ids, scores, rank of cited chunks, fraction of queries below score threshold	Spot corpus gaps and drift
Generation	Tokens in and out, finish reason, groundedness verdict, abstention rate	Control cost and hallucination
Cost	Dollars per query by component	Budget and optimisation

Signal	What to record	Use
Reliability	Error rate, timeouts, retries, cache hit rate	Alarms and capacity planning
User feedback	Thumbs up or down, flagged answers	Feeds the evaluation set

Build one Grafana or CloudWatch dashboard with these panels and put a screenshot in the README. Set alarms on p95 latency, error rate, and a rising abstention rate (often the first sign of a stale index).

15. Repository structure and API design

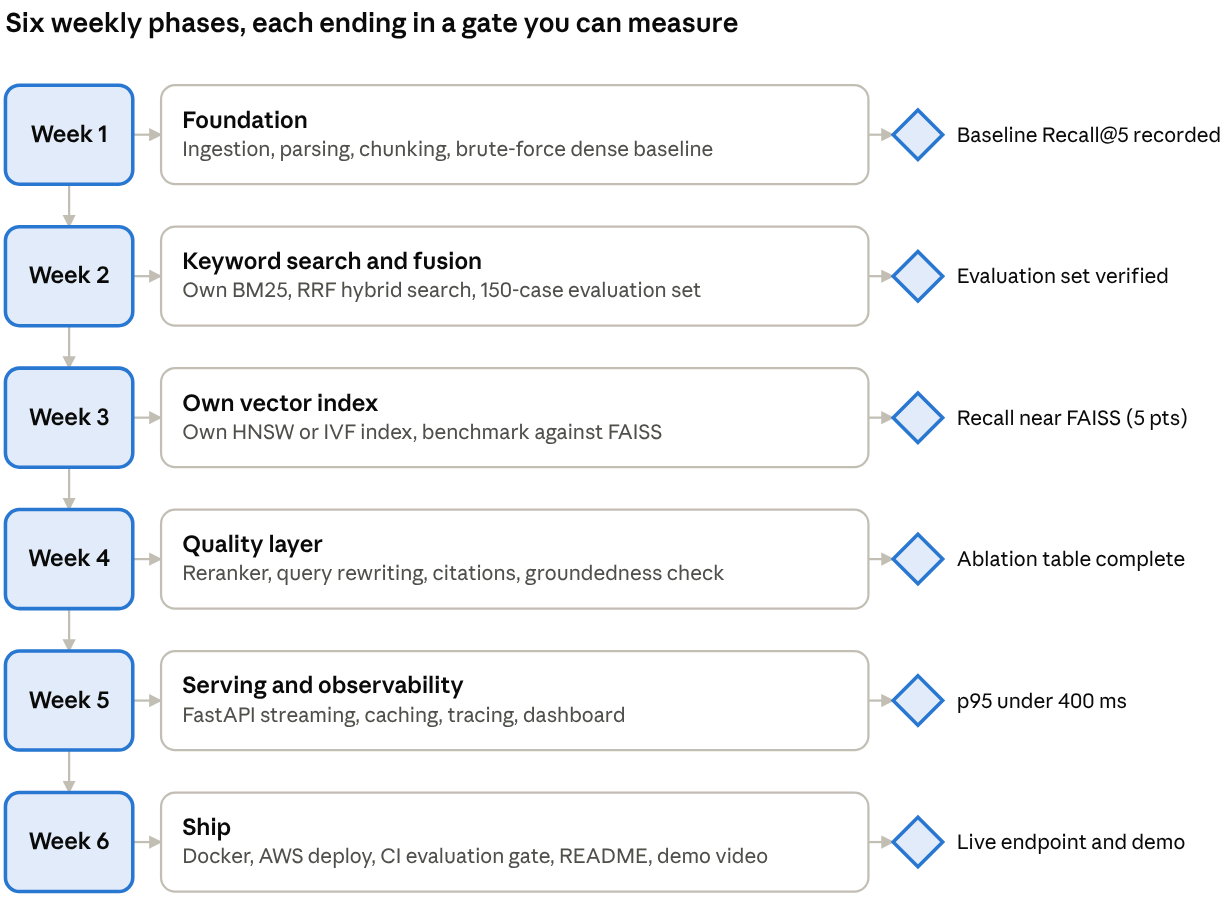
A clean layout lets a reviewer find the interesting code in under a minute.

```
ragforge/
  README.md              architecture diagram, ablation table, quick start
  docs/                  design notes, threat model, scaling note
  src/ragforge/
    ingestion/           loaders, parsers, cleaning, dedup, chunking
    embedding/           model interface, batching, cache
    index/
      bm25.py            own BM25 inverted index
      hnsw.py            own HNSW (and ivf.py, brute_force.py)
      store.py           persistence and versioning
    retrieval/           hybrid search, RRF, filters, parent-child
    rerank/              cross-encoder wrapper
    query/               condense, multi-query, HyDE, routing
    generation/          prompts, context builder, citations, groundedness
    guardrails/          injection filter, PII redaction, ACL checks
    api/                 FastAPI app, schemas, auth, rate limits, SSE
    observability/       tracing, metrics, cost accounting
  eval/
    datasets/            cases in JSONL
    metrics.py           recall, MRR, nDCG, faithfulness
    run_eval.py          ablation runner and report writer
  benchmarks/            own index vs FAISS, load tests
  infra/                 Dockerfile, Terraform or CDK
  tests/                 unit and integration tests
  .github/workflows/     CI with evaluation gate
```

API endpoints

Endpoint	Purpose
POST /ingest	Submit documents or an S3 prefix; returns a job id
GET /ingest/{job_id}	Job status and counts
POST /query	Question, filters, conversation id; returns answer with citations (streamed over SSE when requested)
POST /feedback	Thumbs up or down on a query id
GET /trace/{query_id}	Stage timings and retrieved chunks for debugging
GET /health and GET /metrics	Liveness and Prometheus metrics

16. Six-week roadmap



roadmap · 6 phases, 6 gates

If time runs short, cut stretch features before cutting a gate: a smaller project with all six gates passed beats a bigger one with none. The 400 ms target covers retrieval and reranking on 100k chunks, excluding the LLM call.

17. Risks and mitigations

Risk	Likelihood	Mitigation
Scope too large for six weeks	High	Hold the must-have features first; treat HNSW, reranking tuning and OpenSearch as separable milestones; cut stretch items without guilt

Risk	Likelihood	Mitigation
Own HNSW is buggy or slow	Medium	Test against brute force on small data first; keep FAISS as a fallback and report the gap honestly
Weak evaluation set makes results meaningless	Medium	Verify cases by hand, include unanswerable questions, validate the LLM judge on hand labels
LLM and API costs grow	Medium	Cache aggressively, use a small local model for tests, set a hard monthly budget alarm
Corpus licensing or privacy issues	Low	Use openly published documents only; keep a SOURCES file
Demo breaks during an interview	Medium	Record a short demo video, keep a local docker-compose run as backup
Results look unimpressive	Medium	Report negative results too; a clear explanation of why reranking did or did not help is still a strong signal

18. Resume, interview and Leadership Principles

18.1 Resume bullets (fill the blanks with your real measurements)

- Built RAGForge, a retrieval-augmented generation engine with hand-written BM25 and HNSW indexes, hybrid retrieval and cross-encoder reranking, deployed on AWS (ECS, RDS pgvector, S3, ElastiCache).
- Raised Recall@5 from [baseline] to [final] on a [N]-question evaluation set through hybrid fusion and reranking, at [Y] ms added p95 latency.
- Matched FAISS HNSW within [Z] points of recall while benchmarking latency and memory; documented the trade-offs.
- Cut cost per 1,000 queries by [P] percent with embedding and semantic caching; added a CI gate that blocks merges on retrieval or faithfulness regressions.

Do not write any number you have not measured; interviewers will ask how you got it.

18.2 Questions to prepare

Question	Points to cover
Why hybrid search instead of dense only?	Dense misses exact identifiers and rare terms; BM25 misses paraphrases; show your ablation numbers
How did you choose chunk size?	Precision versus context trade-off, sweep results, parent-child retrieval
HNSW parameters M, efConstruction, ef: what do they do?	Graph connectivity, build quality, search breadth; recall versus latency curve you measured
How do you know the system is good?	Evaluation set construction, metrics, judge validation, CI gate
How do you stop hallucinations?	Retrieval threshold, citations, groundedness check, abstention tests
What if the corpus grows 1000 times?	Sharding, quantisation, async ingestion, scatter-gather, cascade ranking
How do you handle updates and deletes?	Content hashes, incremental re-index, tombstones, blue-green index swap
What is your biggest failure case?	A real failure from your traces, its root cause and fix
What would you change with more time?	Fine-tuned reranker, learned sparse retrieval, agentic multi-step retrieval

18.3 Amazon Leadership Principles mapping

Principle	Evidence from this project
Dive Deep	Wrote BM25 and HNSW yourself and can explain every parameter and failure
Customer Obsession	Defined user questions, abstention behaviour and citations around answer trust

Principle	Evidence from this project
Deliver Results	Shipped a deployed service with measured quality, latency and cost
Invent and Simplify	Replaced framework glue with small, tested components; cache design
Ownership	Added monitoring, alarms, rollback and a threat model without being asked
Bias for Action	Built the baseline first, then improved one measured step at a time
Insist on the Highest Standards	Evaluation gate in CI, honest reporting of negative results

18.4 Portfolio checklist

- [ ] README with architecture diagram, quick start and ablation table
- [ ] Two-minute demo video and a live or recorded deployed endpoint
- [ ] Benchmark plots: recall versus latency for own index and FAISS
- [ ] Evaluation report with per-component results and failure analysis
- [ ] Design notes: threat model and scaling to 100M chunks
- [ ] Short blog post or write-up explaining one surprising finding